

From Single Server to Global Scale

Building Netflix-Style Architecture on AWS

SECTION 1

The Story That Changed Everything

August 2008 - Netflix goes dark for 3 days

August 2008: Netflix Goes Dark

"The most embarrassing moment in Netflix history." - Reed Hastings, CEO

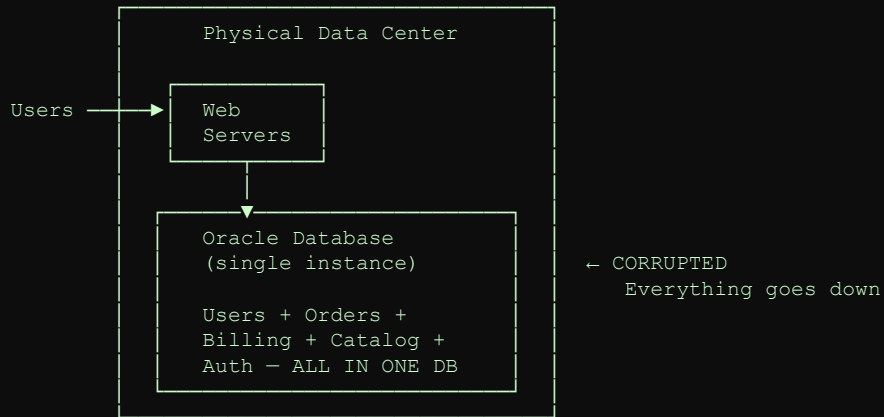
What happened:

A database corruption event hit Netflix's single Oracle database
The entire service went offline - not an hour, not overnight: 3 days
DVD shipments stopped. Subscribers were furious.

Why was it so bad?

Everything ran on one physical data center, one big database
One failure cascaded into total failure - a monolith with no redundancy
There was no failover. There was no recovery plan. There was just: down.

What Broke: The 2008 Netflix Monolith



SINGLE POINT OF FAILURE: one DB dies → entire service dies.
No redundancy. No cloud. No auto-recovery.

The monolith: one failure takes down every feature simultaneously.

The Aftermath: 3 Years to Rebuild on AWS

Netflix's three goals after the outage:

1. No single point of failure - any component can die without taking the service down
2. Scale with demand - handle 10x traffic spikes automatically
3. Global performance - a user in Mumbai gets the same experience as New York

The result today:

250 million subscribers across 190 countries

99.99% uptime - less than 1 hour of downtime per year

700+ independent microservices, each deployed hundreds of times per day

In the next 3 hours, we follow that same journey. Every phase we build, Netflix built.

SECTION 2

Netflix Today: The North Star

Where we're headed - and why it's worth the journey

Netflix at Scale

250M+

Subscribers
across 190 countries

99.99%

Uptime
< 1 hr downtime / year

700+

Microservices
independently deployed

15%

of global internet
traffic at peak

Three Pillars of Netflix Architecture

1. MICROSERVICES - Break the monolith

Every feature is its own independent service. A billing bug can't take down streaming.

2. RESILIENCE - Chaos Engineering

Netflix intentionally breaks their own systems in production every day.

If it can fail, they want to find out now - not during a real outage.

3. GLOBAL SCALE - Open Connect CDN

Netflix built their own CDN embedded inside ISPs worldwide.

Your video doesn't come from a data center - it comes from a box inside your ISP.

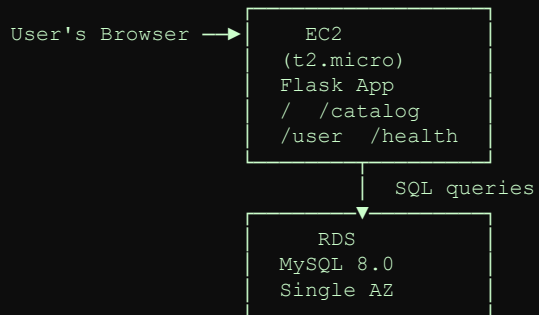
These three themes map to everything we build today.

SECTION 3

Phase 1: Single Server Baseline

EC2 + RDS - Every app starts somewhere

Phase 1 Architecture: EC2 + RDS



Region: us-east-1

- ⚠️ EC2 hardware fault → app is down. No fallback.
- ⚠️ Traffic spike → t2.micro melts at ~500 concurrent users.
- ⚠️ RDS Single-AZ → database failure = data inaccessible.

This is the monolith stage. Fast to build. The first thing we fix.

EC2 + RDS: What You're Actually Using

Amazon EC2

Virtual machine in AWS's data center

You pick size: t2.micro = 1 vCPU, 1 GB RAM

You pick OS: Amazon Linux 2023

SSH in, install your app, run it

Free tier: 750 hrs/month on t2.micro

= 1 instance running 24/7, free

Amazon RDS

Managed database (MySQL, Postgres, Aurora...)

AWS handles: patches, backups, monitoring

You handle: schema, queries, app logic

Free tier: db.t3.micro, 20 GiB storage

Multi-AZ = automatic standby in a second AZ

Always use Multi-AZ in production

The Problem: Single Point of Failure

Scenario 1 — Hardware fault:

AWS's physical server reboots your EC2. App is down. No redundancy.

Scenario 2 — Traffic spike:

10,000 users hit your t2.micro simultaneously.

CPU hits 100%. Responses slow → timeout → crash.

This is exactly what Netflix faced. Their fix:

Multiple servers + automatic failover + automatic scaling.

That is Phase 2.

LAB 1

EC2 + RDS: Launch Your First Server

Duration: 20 min

Goal: Deploy a Flask app on EC2 connected to RDS - your Phase 1 baseline

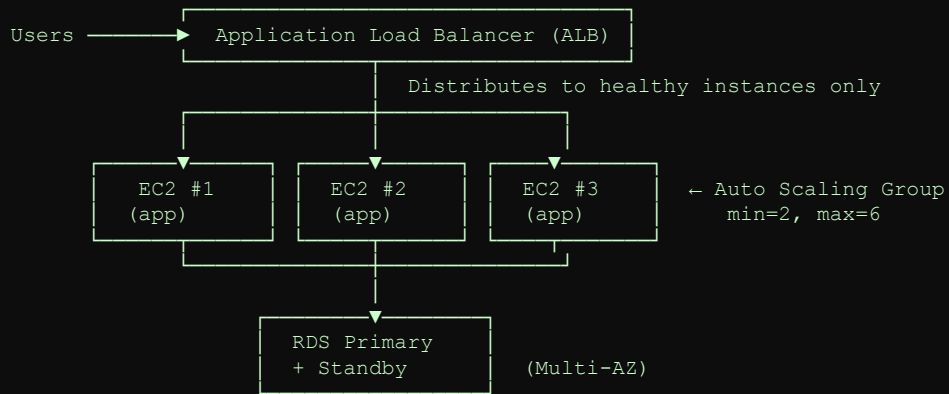
Step-by-step instructions in the recorded lab session

SECTION 4

Phase 2: Resilience & Auto-Scaling

ALB + Auto Scaling Group - Design for failure

Phase 2 Architecture: ALB + Auto Scaling Group



EC2 dies → ALB health check fails → traffic rerouted → ASG launches replacement
CPU > 70% → ASG adds instances | CPU < 30% → ASG removes instances

Application Load Balancer: The Restaurant Host

Analogy:

An ALB is like the host at a busy restaurant - checks which tables are free, seats each arriving guest at an available one. ALB does this with HTTP requests.

How health checks work:

ALB calls GET /health on every instance every 30 seconds

Returns HTTP 200 → healthy, receives traffic

Returns 5xx or times out → removed from rotation automatically



Gotcha:

Use a dedicated /health endpoint, not /. A redirect (301 from /) fails the check.

ALB marks all instances unhealthy. Everything goes down. Use /health → 200 only.

Auto Scaling Group: Never Waste, Never Crash

The fixed-capacity trap:

Instagram Monday 9am traffic = 10× their Tuesday 3am traffic

Provision for peak → you waste 80% of the time

Provision for average → you crash at peak

How ASG solves it:

Define: min=2, max=6, target CPU utilization=70%

CPU > 70% → launch a new EC2 (scale out)

CPU < 30% → terminate an EC2 (scale in)

Netflix at primetime:

200 EC2s → 2,000 EC2s during evening peak → back to 200 overnight

Fully automated. No engineer wakes up at midnight to provision servers.

Chaos Engineering

Netflix's most counterintuitive practice

Chaos Monkey: Breaking Things on Purpose

"If something can break, we want to break it ourselves - before it breaks us."

What Chaos Monkey does:

Randomly terminates EC2 instances in Netflix's production environment

Not in staging. Not in testing. In production. Every day.

Forces every team to build services that survive instance failures

The Simian Army:

Chaos Monkey : kills random instances

Chaos Kong : takes down an entire AWS region

Latency Monkey : injects artificial latency into service calls

Doctor Monkey : checks instance health, removes unhealthy ones

In Lab 2 you run your own Chaos Monkey experiment.

LAB 2

ALB + ASG: Build & Break

Duration: 25 min

Goal: Create ALB + ASG, then simulate Chaos Monkey by terminating an instance and watching auto-recovery

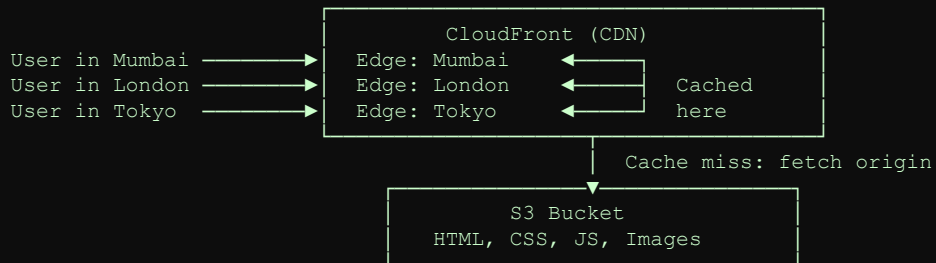
Step-by-step instructions in the recorded lab session

SECTION 5

Phase 3: Global Performance

S3 + CloudFront - Serving a planet

Phase 3 Architecture: S3 + CloudFront CDN



Dynamic API requests → CloudFront passthrough → ALB → ASG → RDS

1st request: CloudFront fetches from S3 (~300ms)

2nd+ request: Served from edge cache (~5ms) X-Cache: Hit

✓ 60% of EC2 load removed. Users worldwide 60× faster.

The Global Latency Problem

The physics problem:

Mumbai → Virginia: 14,000 km round trip = ~140 ms in transit alone

Add server processing: 250-400 ms per request for a distant user

The wasted work problem:

60-70% of every web page is static: images, CSS, JS bundles

These files are identical for every user. Why ship them from Virginia every time?

The fix:

S3: store static files in infinitely-scalable object storage. No EC2 needed.

CloudFront: distribute to 400+ edge locations. Mumbai user → Mumbai edge = 5ms.

Netflix Open Connect: Building Your Own CDN

CloudFront is how most companies solve this. Netflix went further.

What is Open Connect?

- Netflix-designed appliances embedded physically inside ISPs worldwide

- Off-peak hours: Netflix pre-loads popular content onto these boxes

- When you press play: video comes from a box inside your ISP - not from AWS

Why build their own instead of using CloudFront?

- Netflix is 15% of global internet traffic - no third-party CDN optimizes for that

- At 250M subscribers, owning hardware is far cheaper than paying per GB

- They can optimize specifically for adaptive video streaming

What you build in Lab 3 is Open Connect - at your scale.

LAB 3

S3 + CloudFront: Your Own CDN

Duration: 20 min

Goal: Upload static assets to S3, create a CloudFront distribution, verify edge caching via X-Cache header

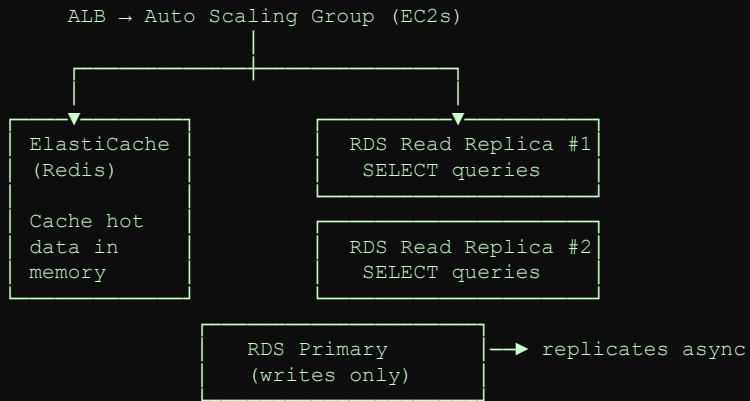
Step-by-step instructions in the recorded lab session

SECTION 6

Phase 4: Database at Scale

ElastiCache + RDS Read Replicas - The read bottleneck

Phase 4 Architecture: ElastiCache + Read Replicas



Cache-aside: Redis hit → microseconds | Redis miss → query RDS + cache result

✓ 80-90% of reads served from cache. Primary handles writes only.

ElastiCache (Redis): The Cache-Aside Pattern

The bottleneck:

Every user action hits the database. Reads vastly outnumber writes.

A single product page = 200+ DB reads. At scale = millions of queries/sec.

The cache-aside pattern:

1. Check Redis first - cache hit → return data in microseconds
2. Cache miss → query RDS, store result in Redis with a TTL (e.g. 5 min)
3. All subsequent requests for that data hit Redis, never touch RDS



Gotcha - Replication Lag:

Replicas are slightly behind the primary (usually ms, can be seconds under heavy writes)

User posts a comment → immediately reloads → might not see it (read hit lagging replica)

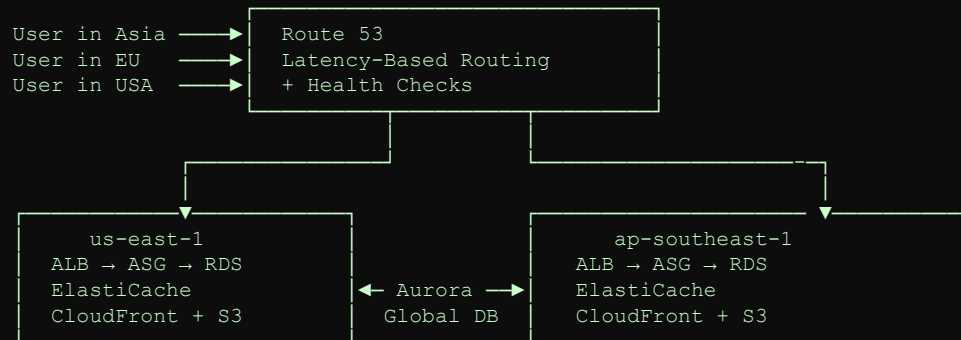
Fix: route reads-after-writes to the primary for that user for a few seconds

SECTION 7

Phase 5: Multi-Region Failover

Route 53 - Surviving a data center going offline

Phase 5 Architecture: Route 53 Multi-Region



us-east-1 fails → Route 53 detects within 30 seconds

→ All traffic autoroutes to ap-southeast-1. No human intervention.

Route 53: More Than Just DNS

What standard DNS does:

Translates 'netflix.com' into an IP address your browser connects to

What Route 53 adds:

Latency Routing: returns the IP of the closest healthy region for each user

→ Tokyo user → ap-northeast-1 | NYC user → us-east-1

Failover: if a region's health check fails, stop routing traffic there

→ 30-second detection window. Automatic. No PagerDuty call needed.



Reality check:

Multi-region is expensive and complex. Most startups don't need it.

Netflix needs it. Understanding it separates senior engineers from junior ones.

Netflix Survived the 2012 AWS Outage

June 2012 - a massive storm hits AWS us-east-1:

Cascading failures across Amazon's Virginia region

Hundreds of companies went dark: Pinterest, Instagram, Heroku, Foursquare...

Netflix?

Most users never noticed.

Why Netflix survived:

Route 53 detected the failure within 30 seconds

Traffic automatically shifted to ap-southeast-1 (Singapore)

Chaos Engineering had trained their systems to handle exactly this

The architecture you learned today is what made that survival possible.

SECTION 8

The Monolith → Microservices Journey

How Netflix broke the thing that broke them

Netflix's Architecture Evolution

2005 - The Monolith

Single Java app, single Oracle DB, single data center. Fast to build. Fatal to fail.

2008 - The Outage. The Decision.

Database corruption. 3 days offline. CEO's call: rebuild everything on AWS.

2009-2012 - The Migration

Team by team, service by service — moved to AWS and split the monolith

Each new service: own codebase, own database, own deployment pipeline

2012 → Today - The Microservices Era

700+ independent services deployed hundreds of times per day

A billing bug cannot break streaming. One team's deploy is isolated.

Chaos Engineering runs continuously. The 2012 outage was survivable.

Monolith vs Microservices

The Monolith

- ✗ One failure = everything fails
- ✗ One team's bug = everyone's problem
- ✗ Scale the whole app or nothing
- ✗ One deploy = risk to all features
- ✓ Simple to start with
- ✓ No network overhead between services
- ✓ Easy to debug locally

Right for: startups, early-stage, small teams

Microservices

- ✓ Billing failure $\neq$ streaming failure
- ✓ Teams deploy fully independently
- ✓ Scale only what needs scaling
- ✓ Smaller blast radius per deploy
- ✗ Network latency between services
- ✗ Distributed systems complexity
- ✗ Harder to trace end-to-end failures

Right for: scale, large teams, high availability

Microservices at Netflix: The Real Picture

What 700+ services looks like:

- API Gateway → routes every request to the right downstream service
- Identity Service → authenticates who you are
- Recommendation → your personalized homepage rows
- Playback Service → licensing, DRM, which server to stream from
- Open Connect → the actual video bytes from ISP-embedded boxes
- Billing Service → subscription management, isolated from everything else

What the app you built has:

- / → serves the whole app (still a monolith today)
- /catalog → seed of a future Catalog Service
- /user → seed of a future Identity/User Service
- /health → the first thing every microservice must have

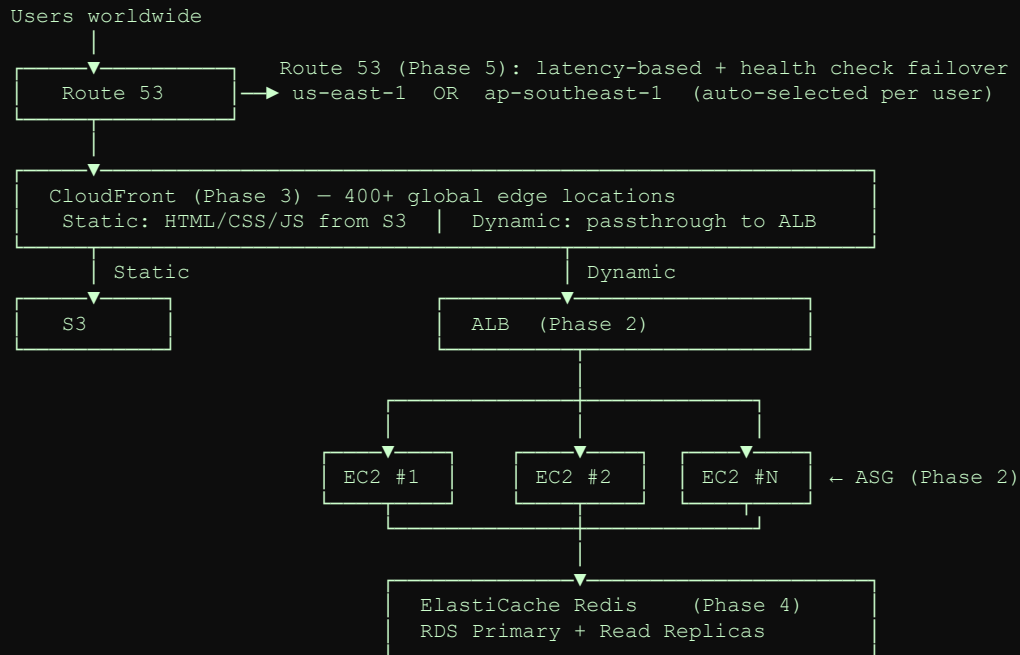
You started with the monolith. Netflix shows you where it goes.

SECTION 9

Bringing It All Together

From EC2 to global scale - the complete picture

Complete Architecture: All 5 Phases



Every Phase, Every Problem, Netflix's Answer

Phase 1 - EC2 + RDS

Problem: get the app running | Netflix: the monolith they started with in 2005

Phase 2 - ALB + Auto Scaling Group

Problem: SPOF + traffic spikes | Netflix: ASG + Chaos Monkey to prove resilience

Phase 3 - S3 + CloudFront

Problem: global latency | Netflix: Open Connect (ISP-embedded CDN appliances)

Phase 4 - ElastiCache + Read Replicas

Problem: DB read bottleneck | Netflix: EVCache (their own distributed Redis grid)

Phase 5 - Route 53 Multi-Region

Problem: regional outage | Netflix: survived 2012 AWS outage, users unaffected

The through-line: Microservices

Monolith → 700+ services. Every phase above applies to each individual microservice.

Career Takeaway

Junior engineer:

Knows how to use AWS services - can launch EC2, configure ALB, create S3 buckets

Senior engineer:

Knows WHY to use each service and WHEN to add each layer

Knows what breaks first and designs around it before it breaks

Can look at any architecture and immediately spot the single points of failure

You now have that mental model.

That puts you ahead of most people interviewing for cloud and backend roles.

What Comes Next

Deepen this architecture:

AWS Well-Architected Framework - free, covers exactly what we built and beyond

Netflix Tech Blog (netflixtechblog.com) - engineers writing about real decisions

"Chaos Engineering" book by Rosenthal et al. - the theory behind Chaos Monkey

Go deeper on individual services:

Containers + ECS/EKS - how Netflix runs microservices at scale

API Gateway - the front door for all 700+ Netflix services

AWS Lambda - serverless, the next layer above what we built today

The AWS Solutions Architect Associate exam:

Everything in today's workshop is covered on the exam. You're already ahead.

Thank You

Questions & Discussion

Remember: run the cleanup steps after the lab sessions!